Modeling Human Activity States Using Hidden Markov Models

Formative 2 (individual report) | Author: [your name] | Student ID: [your ID] | Repository: [GitHub link]

Background and Motivation

Smartphones carry inertial sensors that make it possible to recognise what a person is doing without extra hardware, which supports fitness and step tracking, monitoring of sedentary behaviour, and context-aware services such as inactivity or fall-risk alerts. Raw accelerometer and gyroscope streams are noisy and sequential: an activity lasts seconds to minutes, and people rarely switch between standing, walking and jumping every second. A Hidden Markov Model (HMM) fits this structure naturally, because it separates the hidden activity, which we want to know, from the noisy sensor features, which we observe, and it encodes how likely one activity is to follow another. This report builds a four-state HMM (Still, Standing, Walking, Jumping) from scratch with numpy, trains it on windowed sensor features, and evaluates it on recordings that were never used for training.

Data Collection and Pre-processing

Accelerometer and gyroscope data were recorded with the Sensor Logger app for this individual assignment. The training clips and the continuous multi-activity clip come from an iPhone 11; the two required unseen test clips come from a different phone, an iPhone 11 Pro. Table 1 lists every recording with its native sampling rate (estimated from the timestamps).

Table 1. *Recordings, devices and original sampling rates (individual data, so the table lists devices instead of group members).*

Recording	Label	Role	Device	Native rate (Hz)	Duration (s)
walking_train_01	Walking	train	iPhone 11	100.1	121.5
jumping_train_01	Jumping	train	iPhone 11	100.1	60.0
jumping_train_02	Jumping	train	iPhone 11	100.1	49.6
standing_train_01	Standing	train	iPhone 11	100.1	122.1
still_train_01	Still	train	iPhone 11	100.1	122.1
jumping_test_01	Jumping	unseen test	iPhone 11 Pro	99.8	25.9
still_test_01	Still	unseen test	iPhone 11 Pro	99.8	122.3
multi_test_01	Still, Standing, Jumping, Walking	held-out test	iPhone 11	100.1	41.0

Harmonising the sampling rate. Both phones record at about 100 Hz with slightly irregular timestamps (100.1 Hz and 99.8 Hz). Every signal is linearly interpolated onto an exact 100 Hz grid, passed through a zero-phase FIR anti-alias filter and decimated by two to a common **50 Hz**. 50 Hz keeps content up to 25 Hz, well above walking and jumping cycles (about 1 to 4 Hz), and the anti-alias filter prevents folding of higher-frequency vibration. Three seconds are trimmed from both ends of each clip to remove phone-handling transients, and each clip is split into non-overlapping **10 s chunks** (remainders under 5 s dropped). Windows are cut inside chunks only, and all chunks of one recording stay on the same side of the train/test split. The data had no missing values, no timestamp gaps above 0.01 s, and the same number of accelerometer and gyroscope samples. Sensor Logger writes axes in the order z, y, x, which the loader corrects. The accelerometer is gravity-free (m/s^2); the clip recorded with gravity enabled keeps gravity in a separate channel that is not used.

Leakage check. The timestamps show that the unseen jumping clip was recorded at the same moment as the training clip jumping_train_01 (by the second phone; overlap 25.9 s). To keep the test data unseen, the first 27.4 s of that training clip (the overlap plus a 1 s margin) were removed from training. The test files differ from the training files in the phone used, but all recordings were made on one day within one project, so participant, placement and environment diversity is minimal; this limits how far results can be generalised.

Windowing. Windows are 2 s long (100 samples at 50 Hz) with 50% overlap. Walking steps and jumps repeat at roughly 1.5 to 4 Hz, so a 2 s window holds several cycles and gives an FFT resolution of 0.5 Hz, while still being short enough to follow changes of activity. Each window takes the label of its source recording; windows of the multi-activity clip take the label of the segment containing the window centre, and four windows overlapping the pick-up transition (10 to 12 s) are decoded but not scored.

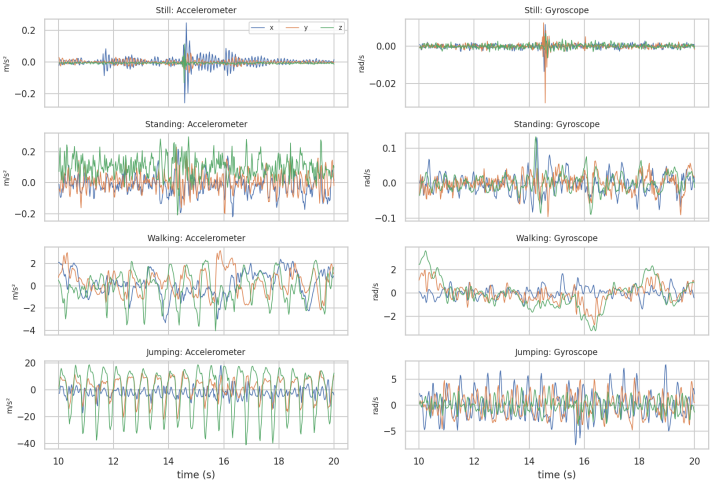


Figure 1. *Raw accelerometer and gyroscope signals (10 s excerpts, 50 Hz after harmonisation) for the four activities. Still and Standing are almost flat; Walking is periodic; Jumping has large, sharp acceleration peaks.*

Features. Eight features per sensor (16 in total), computed per window: time domain, mean of the magnitude, variance, RMS, signal magnitude area (SMA) and mean absolute inter-axis correlation; frequency domain (FFT of the detrended, Hann-windowed magnitude), dominant frequency, spectral energy and normalised spectral entropy. Variance, RMS, SMA and energy differ by up to six orders of magnitude between Still and Jumping, so they are log10-transformed before normalisation. All features are then **Z-scored with a scaler fitted on training windows only** and applied unchanged to the test files; fitting on test data would leak the test distribution into training, and Z-scoring stops large-valued features from dominating the diagonal Gaussians. Variance, RMS and SMA rise with movement intensity (median accelerometer variance 0.0002, 0.0091, 5.1 and 255 for Still, Standing, Walking and Jumping); dominant frequency separates rhythmic from noise-like motion; spectral entropy is high for broadband noise and lower for periodic motion; gyroscope SMA captures body rotation (Figure 2).

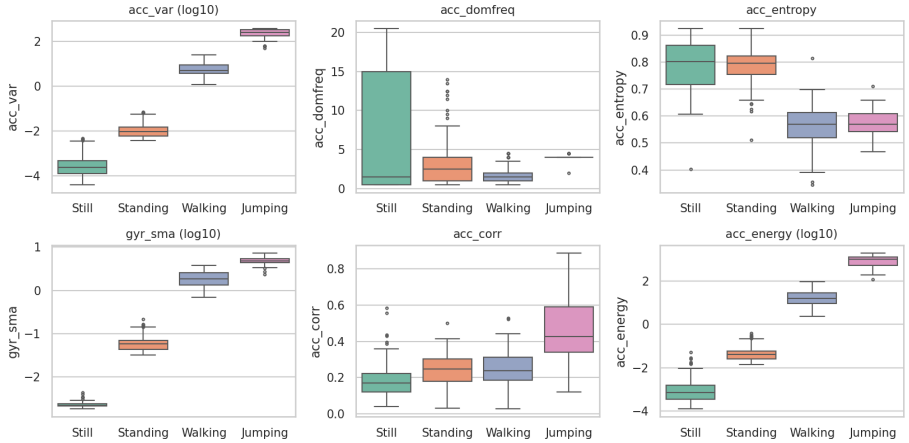


Figure 2. *Distribution of six features per activity (training windows). Variance and gyroscope SMA separate the activities by orders of magnitude; dominant frequency and entropy separate periodic from noise-like motion.*

Table 2. *Placeholder for the four activity videos (to be recorded and linked by the author).*

Activity	Video link
Standing	[add your video link here: video not yet recorded or linked]
Walking	[add your video link here: video not yet recorded or linked]
Jumping	[add your video link here: video not yet recorded or linked]
Still	[add your video link here: video not yet recorded or linked]

HMM Setup and Implementation

Model. The four hidden states are the activities. Each state emits the 16-dimensional Z-scored feature vector from a Gaussian with diagonal covariance (variance floor 0.01 in Z units, to prevent collapse on near-constant features such as Still). The parameters are the initial distribution *pi*, the 4 x 4 transition matrix *A* and the emission means and variances *B*. Everything is implemented in numpy (src/hmm.py, class GaussianHMM): forward and backward passes in log-space with log-sum-exp (no underflow), Viterbi decoding in log-space, and Baum-Welch over several sequences, with a transition floor of 0.001 as smoothing.

Supervised vs unsupervised. Because labels exist, *pi*, *A*, and the emission means and variances are first initialised from the labelled training sequences (supervised). Baum-Welch then re-estimates them from the observations alone,

ignoring the labels (unsupervised refinement). Training stops when the total log-likelihood improves by less than 1e-4 (cap of 200 iterations). From the supervised start it stopped after 2 iterations (log-likelihood 4840.1, final change 1e-08): the classes are so well separated that the labelled start is already a fixed point of EM. To show that EM does converge on its own, an additional unsupervised run from random initialisations was made (5 restarts): 3 restarts reached the same log-likelihood (4840.1) within 5 iterations and the other 2 stalled in a much worse local optimum (-8493.4). After naming the learned states by majority vote on training windows, the best run gives the same held-out accuracy (Figure 6).

Training sequences. No recording contains more than one activity, so a transition matrix estimated from single recordings would be an uninformative near-identity matrix. Instead 40 training sequences (2780 windows) were built by concatenating eight randomly ordered 10 s training chunks, each chunk of a different activity from the previous one (seeded). The off-diagonal transition probabilities therefore reflect this construction (a switch every 9 windows on average, destination chosen at random), not measured behaviour. Sanity tests (run in the notebook) confirm that the forward pass and Viterbi path match brute-force enumeration on a small model, that the rows of A sum to 1, and that the log-likelihood never decreases across EM iterations.

Why Viterbi and Baum-Welch. The task is to recover the most likely sequence of activities from noisy feature sequences, which is exactly what Viterbi decoding returns; it uses the transition matrix to favour persistent activities, so isolated noisy windows are less likely to flip the label, and this directly affects sensitivity (fewer missed windows of an activity) and specificity (fewer false alarms for other activities). Baum-Welch fits the parameters by maximising the likelihood of the observations, handles the fact that state assignments are only softly known, and its convergence criterion gives a principled stopping rule. Overall accuracy summarises how many windows Viterbi labels correctly, while one-vs-rest sensitivity and specificity show which activities are missed or confused.

Results and Interpretation

All test recordings are decoded with Viterbi, each as its own sequence, and none was used for training, scaling or tuning (split by recording, not by window). The two required unseen files (jumping_test_01 and still_test_01, iPhone 11 Pro) give 133 windows; they contain only Still and Jumping, so Walking and Standing have no test windows and their sensitivity is undefined (Table 3). The continuous multi-activity clip, also held out, adds Standing and Walking (Table 4). Its segment boundaries were read from the signal energy and confirmed by the author, and the scored windows number only 30, so per-class rates are rough estimates.

Table 3. *One-vs-rest metrics on the two required unseen files. Overall multi-class accuracy: 0.962 (133 windows).*

Activity	Samples (windows)	Sensitivity	Specificity	Accuracy (one-vs-rest)
Still	115	0.957	1.000	0.962
Standing	0	undefined	0.962	0.962
Walking	0	undefined	1.000	1.000
Jumping	18	1.000	1.000	1.000

Table 4. *One-vs-rest metrics on all held-out recordings (two unseen files plus the multi-activity clip, 163 windows). Overall accuracy: 0.920. Per file: jumping_test_01 1.000, still_test_01 0.957, multi_test_01 0.733.*

Activity	Samples (windows)	Sensitivity	Specificity	Accuracy (one-vs-rest)
Still	120	0.958	1.000	0.969
Standing	8	1.000	0.968	0.969
Walking	6	1.000	0.949	0.951
Jumping	29	0.724	1.000	0.951

Table 5. *Overall accuracy of the main model and of reference variants. The frame-wise classifier uses the same Gaussians but ignores transitions.*

Model	2 unseen files	All held-out	Multi-activity only
Viterbi + Baum-Welch (main)	0.962	0.920	0.733
Viterbi, supervised init only	0.962	0.920	0.733
Frame-wise Gaussian (no transitions)	0.962	0.926	0.767
Unsupervised Baum-Welch (random init)	0.962	0.920	-

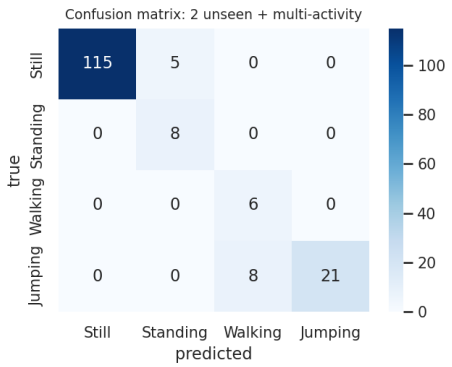


Figure 3. *Confusion matrix on all held-out windows (rows: true activity).*

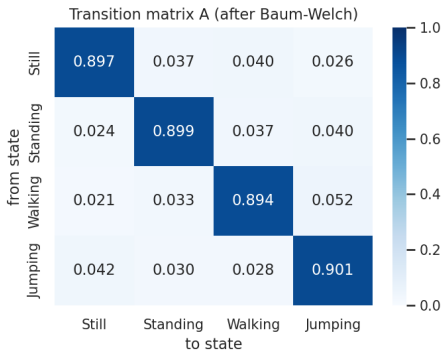


Figure 4. *Transition matrix A after Baum-Welch (annotated).*

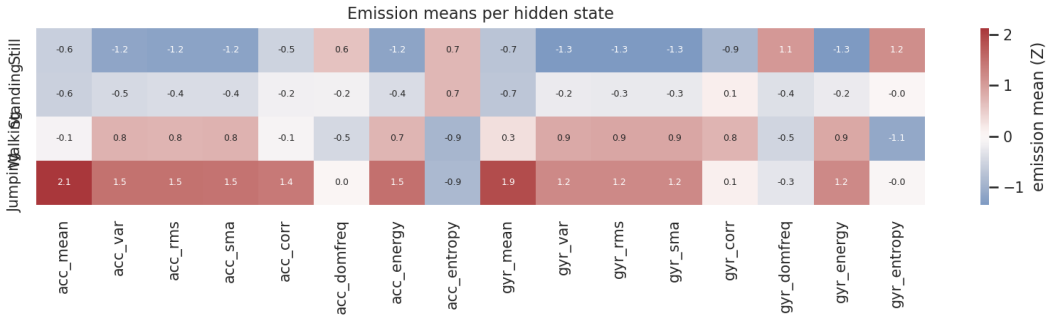


Figure 5. *Emission means of the four states for the 16 Z-scored features. The states differ mainly in the variance, RMS, SMA and energy features, and Still and Standing differ only by a few tenths of a Z-unit in the low-energy features.*

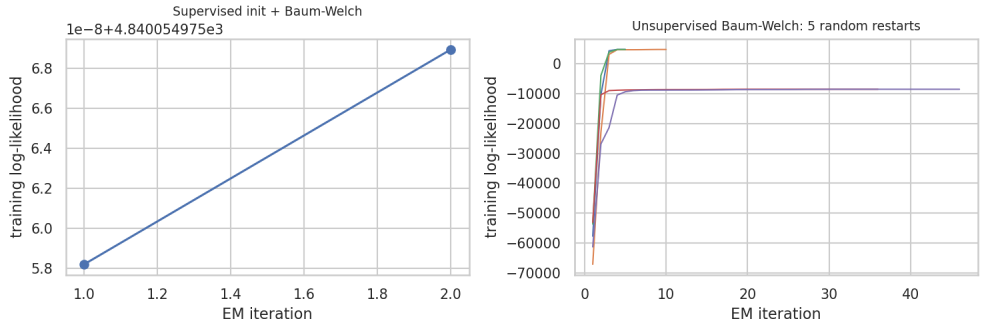


Figure 6. *Baum-Welch log-likelihood. Left: supervised initialisation, already converged. Right: unsupervised EM from five random initialisations; it increases monotonically and stops when the change falls below 1e-4.*

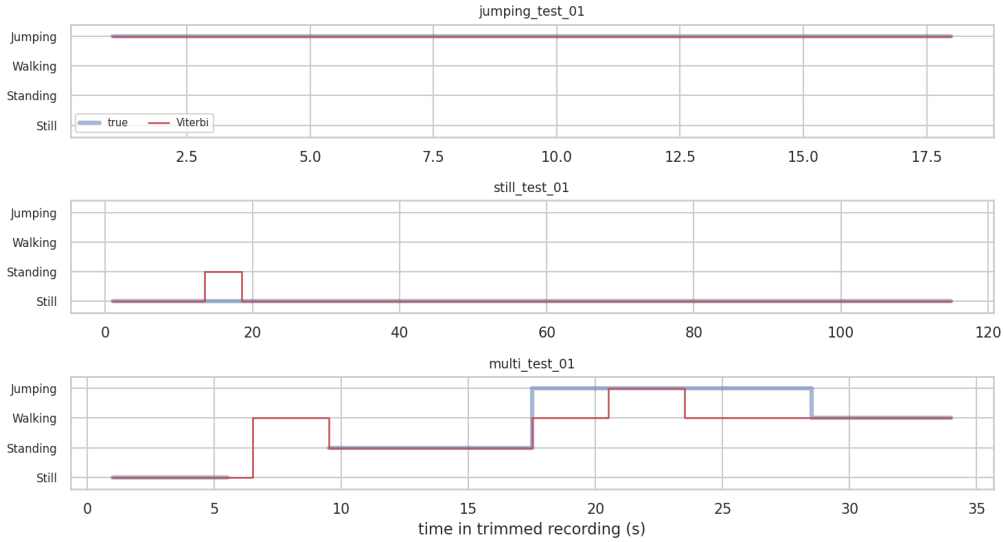


Figure 7. *Viterbi-decoded versus true activity for the three held-out recordings (the gap in the multi-activity line is the unscored pick-up transition).*

Discussion and Conclusion

Easiest and hardest activities. Jumping in the unseen jumping file was decoded perfectly (18 of 18 windows), and Still in the unseen still file almost perfectly (110 of 115); Walking and Standing in the multi-activity clip were all correct, though from only 6 and 8 windows. The misses are informative. The five Still windows decoded as Standing are consecutive (13 to 18 s of the recording): that stretch has a short burst of vibration with accelerometer variance up to 0.028, above any training Still window (maximum 0.0046) and within the Standing range, so this is arguably a genuine disturbance in a clip labelled Still rather than a random model error. It does show that Still versus Standing is the thinnest margin: both are low-energy and noise-like, and only the magnitude of tiny movements separates them. The largest failure is Jumping in the multi-activity clip: only 3 of its 11 windows were recognised and 8 were decoded as Walking. The jumps there are far gentler than in training (median accelerometer variance 43.9 against 334.2, 196.3 and 234.1 in the other jumping recordings; gyroscope SMA 1.62 against about 4.86), so they fall between the training Walking and Jumping distributions. This is a behaviour or placement shift; the model was not adjusted after seeing it.

Transition probabilities. The learned matrix has a diagonal of 0.89 to 0.90 and off-diagonal entries of 0.02 to 0.05. The high diagonal is realistic (an expected stay of about 10 windows, i.e. 10 s at a 1 s window step, matching the persistence of activities). However, the near-uniform off-diagonal entries are an artefact of the training sequences (random chunk order), so they should not be read as real preferences such as Walking leading to Jumping more often than Still does. Real transition structure would need continuous recordings of natural behaviour.

Noise and sampling rate. The unseen clips come from a different phone, and its still recording has a higher noise floor (median window variance 0.0006 against 0.0002 in training), yet this stays far below Standing (median 0.0091), so the 50 Hz harmonisation and the log-scaled features absorbed the device difference. For Still windows the dominant-frequency feature is driven by noise and carries little information, which the large Still variance in that dimension reflects. At 50 Hz and a 2 s window the frequency resolution is 0.5 Hz, which is coarse for the 1 to 2 Hz walking cadence.

Does it generalise? Only partly, and the evidence is limited. The pooled accuracy (0.920) is dominated by the long Still recording; the held-out set is small, windows overlap by 50%, and the Walking, Standing and multi-activity results come from a single short clip. The main HMM did not beat a frame-wise Gaussian classifier that ignores transitions (0.926 against 0.920 overall, 0.767 against 0.733 on the multi-activity clip, a difference of one window), which is expected because the test recordings are almost entirely single-activity and the transition matrix is constructed rather than learned. All recordings come from one project on a single day with two phones, and the multi-activity boundaries were chosen from the same signal energy that the features measure, which can flatter that clip's score.

Improvements. (1) More data: several sessions, participants, phone placements and jump intensities, with continuous recordings of natural activity sequences to estimate A from real behaviour. (2) Better features: per-axis and jerk statistics, autocorrelation or wavelet features, and a longer window or per-user normalisation for the low-frequency Still/Standing contrast. (3) Additional sensors such as the magnetometer, barometer or gravity direction to distinguish Standing from Still by orientation. (4) Model variants: Gaussian-mixture emissions for activities with several intensity levels, full-covariance emissions, hidden semi-Markov models for explicit duration, or a hierarchical HMM.

Conclusion. A from-scratch Gaussian HMM with supervised initialisation and Baum-Welch refinement recovers the four activities with 96.2% accuracy on the two required unseen recordings and 92.0% on all held-out data. The results are encouraging for clearly different activities, but jump intensity, the Still/Standing margin and the small, single-project test set limit what can be claimed about generalisation.